# PROJECT REPORT

# ON

# FITNESS ACTIVITY ANALYZER

Submitted in partial fullfillment of the requirements for the award of the degree of

## MASTER OF COMPUTER APPLICATIONS(MCA)

## IN

## DATA SCIENCE


**SUBMITTED TO:**
**Mrs.Pawandeep Sharma**
**Assistant Professor**
**E2916**

**SUBMITTED BY:**
**Nidhi Maurya**
**26MCD10012**
**26MCD-1(A)**






**UNIVERSITY INSTITUTE OF COMPUTING**
**CHANDIGARH UNIVERSITY**
**Gharuan, Mohali,Punjab**


**SESSION: 2026-2027**

UNIVERSITY INSTITUTE OF COMPUTING

Discover. Learn. Empower.

NAAC GRADE A+

Accredited University

# CERTIFICATE

This is to certify that the project titled “Fitness Activity Analyzer Using Python, Object-Oriented Programming and Matplotlib” is a project work prepared for the subject Python Programming as part of the Master of Computer Applications (MCA) in Data Science during the academic session 2026–2027.

The project demonstrates the use of Python programming, Object-Oriented Programming concepts, file handling, fitness record management, basic statistical calculations, and data visualization using Matplotlib.

The application allows users to add, view, search, update, and delete daily fitness records. It also calculates fitness summaries and presents activity information through graphical representations.

**Project Guide:**

Mrs. Pawandeep Sharma(Assistant Prof. UIC-CU)

**Head Of Department:**

Amanpreet Kaur Sandhu

**Date: ……**

# TABLE OF CONTENTS


| Sr.No. | Contents |
|---|---|
| 1 | Problem Statements |
| 2 | Objectives |
| 3 | technologies / Software Requirements |
| 4 | System Features |
| 5 | Source Code and Outputs |
| 6 | Future Scope |
| 7 | Learning Outcomes |
| 8 | References |

# 1. Problem Statement

Maintaining daily fitness records manually can be difficult because users need to track different activities, such as the number of steps walked, exercise duration, and estimated calories burned. Manual record keeping can also make it difficult to compare daily activities, calculate averages, and understand fitness progress over time.

Therefore, there is a need for a simple computerized application that can record, manage, and analyze daily fitness information.

The **Fitness Activity Analyzer** is a Python-based, menu-driven application developed to solve this problem. It allows users to enter daily fitness information, save records in a CSV file, search and update existing records, calculate summary statistics, and visualize fitness data using graphs created with Matplotlib.

The project demonstrates how Python programming and data visualization can be used to organize fitness information and make it easier to understand.

# 2. Objectives

The main objectives of this project are:

1. To develop a menu-driven application for managing daily fitness activities.
2. To apply Object-Oriented Programming concepts using classes, objects, and methods.
3. To record daily steps, exercise duration, and estimated calories burned.
4. To store fitness records using CSV file handling.
5. To provide options to add, view, search, update, and delete fitness records.
6. To calculate total and average steps, exercise duration, and estimated calories.
7. To validate user input and reduce common data-entry errors.
8. To create graphical representations of fitness data using Matplotlib.
9. To improve understanding of Python programming, file handling, and data analysis.

# 3. Technologies / Software Requirements:

1. Programming Language: Python 3
2. Libraries: Matplotlib for data visualization and CSV module for storing and managing fitness records.
3. Development Environment: Visual Studio Code (VS Code).
4. Data Storage: CSV file (`fitness_data.csv`) for saving daily fitness activities.
5. Official Documentation:
    1. Python
    2. Matplotlib
    3. Python CSV Module
    4. Python Object-Oriented Programming
    5. Visual Studio Code
    6. Hardware Requirements: Laptop or computer with Python installed.
    7. Internet Resource: Internet access for installing Matplotlib and referring to official documentation.

# 4. System Features

## 4.1 Add Fitness Activity

The application allows users to enter their daily fitness information, including date, number of steps, exercise duration in minutes, and estimated calories burned. The entered information is stored as a fitness record.

## 4.2 View Fitness Records

The user can view all saved fitness records. This feature helps users review their daily activity information and compare records from different dates.

## 4.3 Search Fitness Activity

The search feature allows users to find a fitness record using its date. It makes retrieving information for a particular day easier.

## 4.4 Update Fitness Record

Users can update an existing record when they need to correct or change previously entered information. This helps maintain accurate records.

### 4.5 Delete Fitness Record

The application provides an option to delete a selected fitness record when it is no longer required.

### 4.6 Fitness Summary and Analysis

The system calculates summary statistics from the saved records. These include total steps, average steps, total exercise duration, average exercise duration, and total estimated calories burned.

These calculations help users understand their recorded physical activity.

### 4.7 Graphical Representation

The application uses Matplotlib to create different types of graphs.

**Daily Steps Bar Chart:** Displays the number of steps recorded for different dates.

**Exercise Duration Line Graph:** Shows changes in exercise duration across recorded dates.

**Estimated Calories Bar Chart:** Compares the estimated calories entered for different days.

**Exercise-Time Pie Chart:** Displays the distribution of recorded exercise time among the available dates.

These graphs help users interpret their fitness information visually.

### 4.8 Data Storage

The application stores records in a CSV file named `fitness_data.csv`. This allows the records to remain available between program executions, provided the file is retained in the correct working folder.

## 5. Source Code:

DSA > Fitness_analyzer.py > ...

```python
import csv
import os
from datetime import datetime
import matplotlib.pyplot as plt

# Class 1: Store one fitness record

class FitnessActivity:
    def __init__(self, date, steps, minutes, calories):
        self.date = date
        self.steps = steps
        self.minutes = minutes
        self.calories = calories


# Class 2: Manage and analyze records

class FitnessAnalyzer:
    def __init__(self):
        self.filename = "fitness_data.csv"
        self.activities = []
        self.load_data()

    # Check date format
    def valid_date(self, date):
        try:
            datetime.strptime(date, "%Y-%m-%d")
            return True
        except ValueError:
            return False

    # Load saved records from CSV
    def load_data(self):
        if not os.path.exists(self.filename):
            return

        try:
            with open(self.filename, "r", newline="") as file:
                reader = csv.DictReader(file)
```

Caption

```
                for row in reader:
                    activity = FitnessActivity(
                        row["date"],
                        int(row["steps"]),
                        int(row["minutes"]),
                        float(row["calories"])
                    )
                    self.activities.append(activity)

            print("Saved fitness data loaded successfully.")

        except (ValueError, KeyError, OSError, csv.Error):
            print("Could not load the CSV file. Check its contents.")

    # Save records permanently
    def save_data(self):
        with open(self.filename, "w", newline="") as file:
            fields = ["date", "steps", "minutes", "calories"]
            writer = csv.DictWriter(file, fieldnames=fields)
            writer.writeheader()

            for activity in self.activities:
                writer.writerow({
                    "date": activity.date,
                    "steps": activity.steps,
                    "minutes": activity.minutes,
                    "calories": activity.calories
                })

    # Find a record by date
    def find_activity(self, date):
        for activity in self.activities:
            if activity.date == date:
                return activity
        return None
```

```
    # Read and validate fitness values
    def get_values(self):
        try:
            steps = int(input("Enter steps walked: "))
            minutes = int(input("Enter exercise minutes: "))
            calories = float(
                input("Enter estimated calories burned: ")
            )

            if steps < 0 or minutes < 0 or calories < 0:
                print("Values cannot be negative.")
                return None

            return steps, minutes, calories

        except ValueError:
            print("Please enter valid numeric values.")
            return None

    # Add a new record
    def add_activity(self):
        print("\n--- Add Fitness Record ---")
        date = input("Enter date (YYYY-MM-DD): ").strip()

        if not self.valid_date(date):
            print("Invalid date. Use YYYY-MM-DD format.")
            return

        if self.find_activity(date):
            print("A record for this date already exists.")
            return

        values = self.get_values()
        if values is None:
            return
```

```
        steps, minutes, calories = values
        self.activities.append(
            FitnessActivity(date, steps, minutes, calories)
        )
        self.save_data()
        print("Record added and saved successfully!")

    # Display all records
    def view_activities(self):
        print("\n--- All Fitness Records ---")

        if not self.activities:
            print("No records found.")
            return

        print(
            f"{'Date':<15}{'Steps':<12}"
            f"{'Minutes':<12}{'Calories':<12}"
        )

        for a in sorted(self.activities, key=lambda x: x.date):
            print(
                f"{a.date:<15}{a.steps:<12}"
                f"{a.minutes:<12}{a.calories:<12.1f}"
            )

    # Search by date
    def search_activity(self):
        date = input("Enter date to search (YYYY-MM-DD): ").strip()
        activity = self.find_activity(date)

        if activity:
            print("\nRecord found:")
            print("Date:", activity.date)
            print("Steps:", activity.steps)
            print("Exercise minutes:", activity.minutes)
            print("Estimated calories:", activity.calories)
```

```

    # Update a record
    def update_activity(self):
        date = input("Enter date to update (YYYY-MM-DD): ").strip()
        activity = self.find_activity(date)

        if not activity:
            print("Record not found.")
            return

        print("Enter the new values:")
        values = self.get_values()

        if values is None:
            return

        activity.steps, activity.minutes, activity.calories = values
        self.save_data()
        print("Record updated successfully!")

    # Delete a record
    def delete_activity(self):
        date = input("Enter date to delete (YYYY-MM-DD): ").strip()
        activity = self.find_activity(date)

        if not activity:
            print("Record not found.")
            return

```

```
182
183         if confirm == "y":
184             self.activities.remove(activity)
185             self.save_data()
186             print("Record deleted successfully.")
187         else:
188             print("Deletion cancelled.")
189
190     # Display statistics
191     def show_summary(self):
192         print("\n--- Fitness Summary ---")
193
194         if not self.activities:
195             print("No records available.")
196             return
197
198         total_steps = sum(a.steps for a in self.activities)
199         total_minutes = sum(a.minutes for a in self.activities)
200         total_calories = sum(a.calories for a in self.activities)
201         count = len(self.activities)
202
203         average_steps = total_steps / count
204         average_minutes = total_minutes / count
205
206         most_active = max(self.activities, key=lambda a: a.steps)
207
208         print("Number of recorded days:", count)
209         print("Total steps:", total_steps)
210         print(f"Average steps per recorded day: {average_steps:.2f}")
211         print("Total exercise minutes:", total_minutes)
212         print(f"Average exercise minutes per day: {average_minutes:.2f}")
213         print(f"Total estimated calories burned: {total_calories:.1f}")
214         print(
215             f"Most active day: {most_active.date} "
216             f"({most_active.steps} steps)"
217         )
218
```

```
223             return
224
225         records = sorted(self.activities, key=lambda x: x.date)
226         dates = [a.date for a in records]
227         steps = [a.steps for a in records]
228         minutes = [a.minutes for a in records]
229         calories = [a.calories for a in records]
230
231         fig, axes = plt.subplots(2, 2, figsize=(13, 9))
232
233         # Graph 1: Steps by date
234         axes[0, 0].bar(dates, steps)
235         axes[0, 0].set_title("Daily Steps")
236         axes[0, 0].set_ylabel("Steps")
237
238         # Graph 2: Exercise duration
239         axes[0, 1].plot(dates, minutes, marker="o")
240         axes[0, 1].set_title("Exercise Duration")
241         axes[0, 1].set_ylabel("Minutes")
242
243         # Graph 3: Estimated calories
244         axes[1, 0].bar(dates, calories)
245         axes[1, 0].set_title("Estimated Calories Burned")
246         axes[1, 0].set_ylabel("Calories")
247
248         # Graph 4: Activity distribution
249         axes[1, 1].pie(
250             minutes,
251             labels=dates,
```

```
256
257         for ax in [axes[0, 0], axes[0, 1], axes[1, 0]]:
258             ax.tick_params(axis="x", rotation=45)
259             ax.grid(axis="y", linestyle="--", alpha=0.4)
260
261         fig.suptitle("Fitness Activity Analysis", fontsize=16)
262         fig.tight_layout()
263         plt.show()
264
265     # Main menu
266     def run(self):
267         while True:
268             print("\n====== ADVANCED FITNESS ANALYZER ======")
269             print("1. Add Fitness Record")
270             print("2. View All Records")
271             print("3. Search Record")
272             print("4. Update Record")
273             print("5. Delete Record")
274             print("6. View Fitness Summary")
275             print("7. Show Graphs")
276             print("8. Exit")
277
278             choice = input("Enter your choice (1-8): ").strip()
279
280             if choice == "1":
281                 self.add_activity()
282             elif choice == "2":
283                 self.view_activities()
284             elif choice == "3":
285                 self.search_activity()
286             elif choice == "4":
287                 self.update_activity()
288             elif choice == "5":
289                 self.delete_activity()
290             elif choice == "6":
291                 self.show_summary()
292             elif choice == "7":
293                 self.show_graphs()
294             elif choice == "8":
```

```
294             elif choice == "8":
295                 print("Thank you for using Fitness Analyzer!")
296                 break
297             else:
298                 print("Invalid choice. Enter a number from 1 to 8.")
299
300
301 # Start the application
302 if __name__ == "__main__":
303     analyzer = FitnessAnalyzer()
304     analyzer.run()
305
306
```

## Outputs:

```
====== ADVANCED FITNESS ANALYZER ======
1. Add Fitness Record
2. View All Records
3. Search Record
4. Update Record
5. Delete Record
6. View Fitness Summary
7. Show Graphs
8. Exit
Enter your choice (1-8): 1

--- Add Fitness Record ---
Enter date (YYYY-MM-DD): 2026-10-26
Enter steps walked: 4000
Enter exercise minutes: 70
Enter estimated calories burned: 120
Record added and saved successfully!
```

```
====== ADVANCED FITNESS ANALYZER ======
1. Add Fitness Record
2. View All Records
3. Search Record
4. Update Record
5. Delete Record
6. View Fitness Summary
7. Show Graphs
8. Exit
Enter your choice (1-8): 2

--- All Fitness Records ---
Date           Steps       Minutes     Calories
2026-10-25     3000        60          300.0
2026-10-26     4000        70          120.0
```

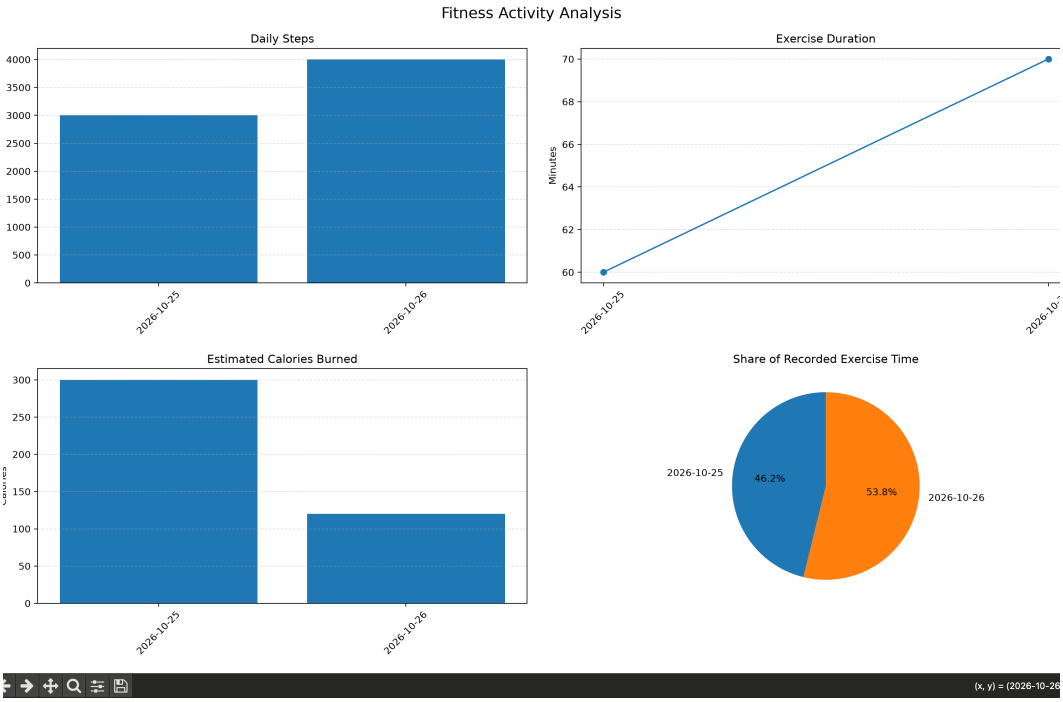
```
====== ADVANCED FITNESS ANALYZER ======
1. Add Fitness Record
2. View All Records
3. Search Record
4. Update Record
5. Delete Record
6. View Fitness Summary
7. Show Graphs
8. Exit
Enter your choice (1-8): 3
Enter date to search (YYYY-MM-DD): 2026-10-26

Record found:
Date: 2026-10-26
Steps: 4000
Exercise minutes: 70
Estimated calories: 120.0
```

Caption

```
====== ADVANCED FITNESS ANALYZER ======
1. Add Fitness Record
2. View All Records
3. Search Record
4. Update Record
5. Delete Record
6. View Fitness Summary
7. Show Graphs
8. Exit
Enter your choice (1-8): 6

--- Fitness Summary ---
Number of recorded days: 2
Total steps: 7000
Average steps per recorded day: 3500.00
Total exercise minutes: 130
Average exercise minutes per day: 65.00
Total estimated calories burned: 420.0
Most active day: 2026-10-26 (4000 steps)
```




# 6. Future Scope

The Fitness Activity Analyzer can be improved in several ways in the future.

## 6.1 Database Integration

The CSV file could be replaced with a database such as SQLite or MySQL to support more structured storage and retrieval of fitness records.

## 6.2 Automatic Activity Tracking

The application could be connected to a supported fitness tracker or mobile health application to collect activity data automatically, if the required integration is available.

## 6.3 Weekly and Monthly Analysis

The system could be extended to calculate weekly and monthly summaries, allowing users to compare their activity over longer periods.

## 6.4 Advanced Data Visualization

Additional graphs and charts could be added to display weekly progress, monthly activity trends, and comparisons between different fitness measurements.

## 6.5 Report Generation

The application could generate PDF or Excel reports containing fitness records, statistical summaries, and graphs.

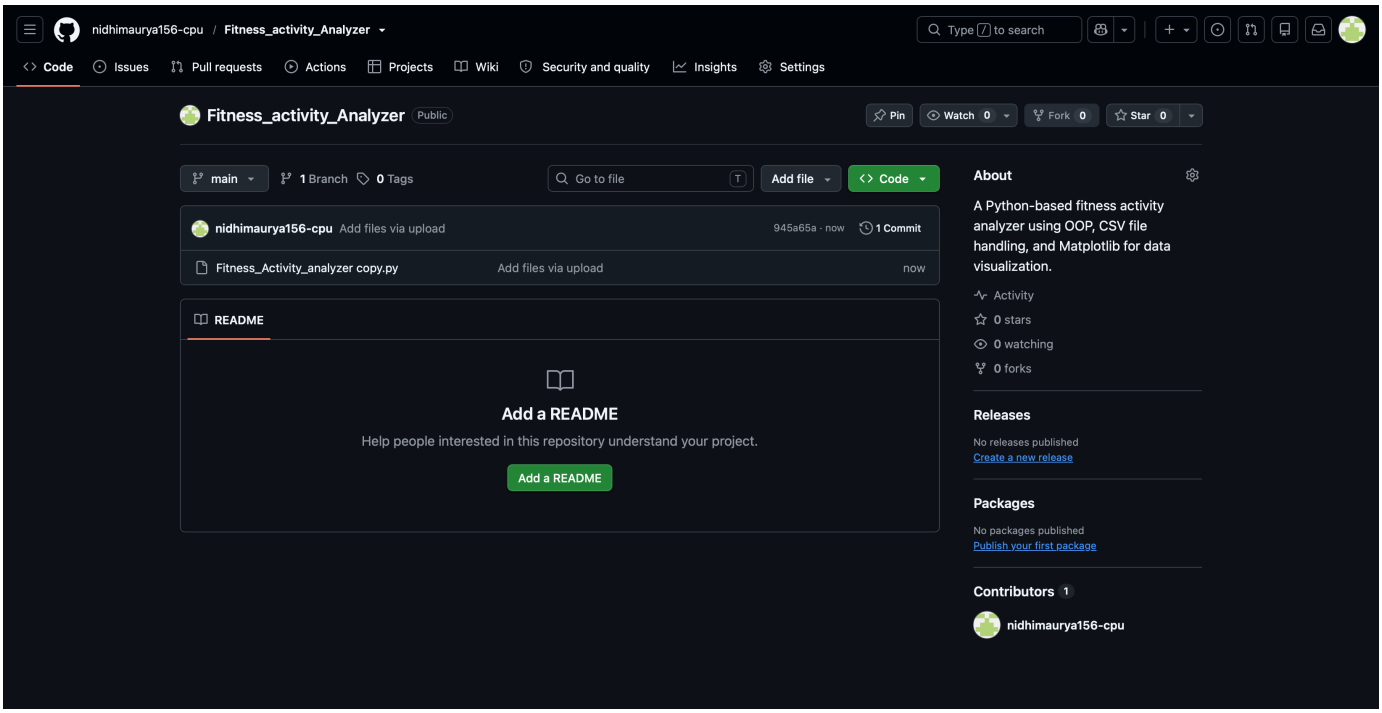
## 6.6 Fitness Goal Tracking

A future version could allow users to set daily step or exercise goals and compare their recorded activity against those goals.

## 6.7 Improved User Interface

The command-line interface could be replaced with a graphical user interface to make the application easier to use.

## Github Repository Overview:




**Github Link:** [https://github.com/nidhimaurya156-cpu/Fitness_activity_Analyzer.git](https://github.com/nidhimaurya156-cpu/Fitness_activity_Analyzer.git)

## 7. Learning Outcomes:

After completing the Fitness Activity Analyzer using Python, OOP, and Matplotlib, the following learning outcomes were achieved:

1. Learned to develop a Python application using Object-Oriented Programming (OOP) concepts such as classes, objects, and methods.
2. Gained experience in storing, retrieving, searching, updating, and deleting fitness activity records using CSV files.
3. Learned to use Python's built-in `csv` module for file handling and data management.
4. Understood how to validate user input and handle invalid data.
5. Learned to use Matplotlib to create bar charts, line graphs, and pie charts for fitness data visualization.

   Developed the ability to calculate total steps, average steps, exercise duration, and estimated calories burned.

7. Improved problem-solving, logical thinking, and Python programming skills.

   Understood how data visualization helps users analyze daily fitness activities and monitor their progress.

# 8. References

Python Official Documentation: https://docs.python.org/3/

- Python Classes Tutorial: https://docs.python.org/3/tutorial/classes.html
- Python CSV Module: https://docs.python.org/3/library/csv.html

Matplotlib Official Documentation: https://matplotlib.org/stable/

Visual Studio Code Documentation: https://code.visualstudio.com/docs